

Project Title: Hospital Management System

Abstract: This report presents a SQL-based Hospital Management System (HMS) designed to replace inefficient manual administrative processes. The system utilizes a normalized relational database to manage core entities like patients, doctors, appointments, and billing, ensuring data integrity. Functionality is driven by diverse SQL queries, while stored procedures and triggers are implemented to automate tasks and enhance operational efficiency. It successfully centralizes patient records, streamlines scheduling, and maintains consistency across financial and medical data. Ultimately, the HMS improves overall hospital performance and elevates the quality of patient care.

1. Introduction

The Hospital Management System (HMS) is a comprehensive and integrated system designed to manage the operations of a hospital in a more efficient and reliable manner. By using database management principles, the system organizes patient records, doctor schedules, medical history, appointments, billing, staff details, medicines, and pharmacy transactions. The primary goal of this project is to reduce manual work, minimize errors, and provide a centralized system that improves hospital service quality and patient care.

2. Objectives

The objectives of the Hospital Management System are:

- To store and manage patient details, medical history, and contact information.
- To schedule and manage doctor appointments in a structured way.
- To maintain accurate medical records including diagnosis, treatment, and prescriptions.
- To streamline billing processes and track payment status.
- To manage hospital staff and their responsibilities effectively.
- To maintain medicine stock and pharmacy transactions.
- To provide real-time access to information for doctors, staff, and administrators.
- To reduce redundancy, ensure data integrity, and improve overall hospital service.

3. Database Design and Table Explanation

The database for the Hospital Management System consists of multiple tables, each focusing on specific aspects of the hospital's operations. The tables are organized to follow best practices for data normalization, ensuring efficiency and preventing data redundancy. Below is an overview of the primary tables:

Patients Table: Stores essential information about patients such as name, contact details, medical history, and date of birth. The `patient_id` is the primary key, making each patient unique. This table links to appointments, medical records, and billing.

Doctors Table: Holds information about doctors including name, specialty, and contact details. The `doctor_id` is the primary key and links to patient appointments and medical records.

Appointments Table: Records patient appointments with doctors, including appointment date, time, purpose, and status. Links patients and doctors through `patient_id` and `doctor_id`.

Medical Records Table: Stores the diagnosis, treatment, and prescriptions for each patient visit. It links to the Appointments and Doctors tables to record what happened during an appointment.

Billing Table: Manages financial information for appointments, such as the total amount, payment status, and insurance provider. It links to patients and appointments to track payments and billing statuses.

Medicine Table: Tracks medicines in the hospital's pharmacy, including medicine name, type, dosage, stock quantity, and expiry date. The `medicine_id` serves as the primary key.

Pharmacy Table: Links patients to the medicines they've been prescribed, storing the `medicine_id`, `patient_id`, quantity, and prescription date. This helps track medication dispensing.

4. SQL Queries and Operations

Below is a detailed explanation of the SQL queries used in the Hospital Management System. These queries perform various operations related to patient management, appointments, billing, stored procedures, and triggers

Get All Table Names in the Database

```
SELECT table_name
FROM information_schema.tables
WHERE table_type = 'BASE TABLE';
```

This query retrieves a list of all the tables in the database. The `information_schema.tables` is a system view that provides metadata about all the tables in the database. The condition `WHERE table_type = 'BASE TABLE'` filters out any views and returns only the base tables (actual data tables).

Show All Records from the Patients Table

```
SELECT first_name, last_name FROM Patients;
```

This query selects only the `first_name` and `last_name` columns from the `Patients` table. It is useful when you only need to retrieve the names of all patients without any other information.

	first_name	last_name
1	Isabella	Anderson
2	Daniel	Brown
3	Emily	Davis
4	John	Doe
5	Michael	Johnson
6	Olivia	Martinez
7	Sophia	Moore
8	Jane	Smith
9	David	Taylor
10	James	Wilson

Show Records from the Patients Table Ordered by Last Name

```
SELECT * FROM Patients ORDER BY last_name ASC;
```

Results Messages										
	patient_id	first_name	last_name	date_of_birth	gender	contact_number	address	email	medical_history	created_at
1	10	Isabella	Anderson	2005-12-01	F	555-4680	741 Birch St, Springfield, IL	isabella.anderson@email.com	None	2025-09-06 19:22:18.133
2	5	Daniel	Brown	1990-03-25	M	555-6543	654 Maple St, Springfield, IL	daniel.brown@email.com	Migraine	2025-09-06 19:22:18.133
3	4	Emily	Davis	2000-11-30	F	555-4321	321 Birch St, Springfield, IL	emily.davis@email.com	None	2025-09-06 19:22:18.133
4	1	John	Doe	1985-05-15	M	555-1234	123 Elm St, Springfield, IL	john.doe@email.com	Hypertension	2025-09-06 19:22:18.133
5	3	Michael	Johnson	1975-02-10	M	555-8765	789 Pine St, Springfield, IL	michael.johnson@email.com	Diabetes	2025-09-06 19:22:18.133
6	6	Olivia	Martinez	1980-07-10	F	555-9876	987 Cedar St, Springfield, IL	olivia.martinez@email.com	Arthritis	2025-09-06 19:22:18.133
7	8	Sophia	Moore	1995-04-05	F	555-2468	753 Oak St, Springfield, IL	sophia.moore@email.com	None	2025-09-06 19:22:18.133
8	2	Jane	Smith	1992-08-22	F	555-5678	456 Oak St, Springfield, IL	jane.smith@email.com	Asthma	2025-09-06 19:22:18.133
9	9	David	Taylor	1983-09-20	M	555-3579	369 Pine St, Springfield, IL	david.taylor@email.com	High cholesterol	2025-09-06 19:22:18.133
10	7	James	Wilson	1965-01-14	M	555-1357	159 Elm St, Springfield, IL	james.wilson@email.com	Heart disease	2025-09-06 19:22:18.133

This query retrieves all columns from the Patients table, but it orders the records alphabetically by the last_name column in ascending order (ASC). This is helpful when you need to see the list of patients sorted by their last names.

Count the Total Number of Patients

```
SELECT COUNT(*) AS total_patients FROM Patients;
```

Results	
	total_patients
1	10

This query counts the total number of patients in the Patients table. The COUNT(*) function counts all rows in the table. The result is returned with the alias total_patients, providing a summary of the number of patients in the system.

Count the Number of Patients Grouped by Gender

```
SELECT gender, COUNT(*) AS total_patients FROM Patients
```

```
GROUP BY gender;
```

Results Messages		
	gender	total_patients
1	F	5
2	M	5

This query groups patients by their gender and counts how many patients fall under each gender category. The GROUP BY clause groups rows with the same values in the gender column, and the COUNT(*) function returns the number of patients for each group.

Count the Number of Patients with Specific Medical History

```
SELECT medical_history, COUNT(*) AS total_patients FROM Patients
WHERE medical_history IS NOT NULL
GROUP BY medical_history;
```

	medical_history	total_patients
1	Arthritis	1
2	Asthma	1
3	Diabetes	1
4	Heart disease	1
5	High cholesterol	1
6	Hypertension	1
7	Migraine	1
8	None	3

This query counts the number of patients who have a specific medical history (non-null entries). It uses the WHERE clause to filter out patients without a medical history (medical_history IS NOT NULL). The GROUP BY clause groups the patients by their medical_history, and COUNT(*) counts the number of patients within each group.

Concatenate First and Last Name as Full Name and Show Contact Number and Medical History

```
SELECT first_name + ' ' + last_name AS full_name, contact_number, medical_history FROM
Patients;
```

	full_name	contact_number	medical_history
1	John Doe	555-1234	Hypertension
2	Jane Smith	555-5678	Asthma
3	Michael Johnson	555-8765	Diabetes
4	Emily Davis	555-4321	None
5	Daniel Brown	555-6543	Migraine
6	Olivia Martinez	555-9876	Arthritis
7	James Wilson	555-1357	Heart disease
8	Sophia Moore	555-2468	None
9	David Taylor	555-3579	High cholesterol
10	Isabella Anderson	555-4680	None

This query concatenates the first_name and last_name columns into a single column called full_name using the + operator. It also retrieves the contact_number and medical_history columns from the Patients table. This query is useful for generating a more readable format of patient names.

Join Patients Table with Appointments Table to Get Patient Details Along with Appointment Dates

```
SELECT p.first_name, p.last_name, a.appointment_date, a.appointment_time
FROM Patients p
INNER JOIN Appointments a ON p.patient_id = a.patient_id;
```

	first_name	last_name	appointment_date	appointment_time
1	John	Doe	2024-11-20	09:00:00.0000000
2	Jane	Smith	2024-11-21	10:00:00.0000000
3	Michael	Johnson	2024-11-22	11:00:00.0000000
4	Emily	Davis	2024-11-23	14:00:00.0000000
5	Daniel	Brown	2024-11-24	15:00:00.0000000
6	Olivia	Martinez	2024-11-25	16:00:00.0000000
7	James	Wilson	2024-11-26	09:30:00.0000000
8	Sophia	Moore	2024-11-27	10:30:00.0000000
9	David	Taylor	2024-11-28	13:00:00.0000000
10	Isabella	Anderson	2024-11-29	14:30:00.0000000

This query performs an INNER JOIN between the Patients and Appointments tables. It links the two tables based on the patient_id column, and retrieves the patient's first_name, last_name, along with the associated appointment_date and appointment_time. This query is used to get the list of patients along with their scheduled appointments

Get the Cartesian Product of Patients and Doctors Tables

```
SELECT * FROM Patients, Doctors;
```

	patient_id	first_name	last_name	date_of_birth	gender	contact_number	address	email	medical_history	created_at	doctor_id	first_name	last_name	specialty	contact_number	ema
1	1	John	Doe	1985-05-15	M	555-1234	123 Elm St, Springfield, IL	john.doe@email.com	Hypertension	2025-09-06 19:22:18.133	1	Dr. Alice	Miller	Cardiology	555-1122	alice
2	2	Jane	Smith	1992-08-22	F	555-5678	456 Oak St, Springfield, IL	jane.smith@email.com	Asthma	2025-09-06 19:22:18.133	1	Dr. Alice	Miller	Cardiology	555-1122	alice
3	3	Michael	Johnson	1975-02-10	M	555-8765	789 Pine St, Springfield, IL	michael.johnson@email.com	Diabetes	2025-09-06 19:22:18.133	1	Dr. Alice	Miller	Cardiology	555-1122	alice
4	4	Emily	Davis	2000-11-30	F	555-4321	321 Birch St, Springfield, IL	emily.davis@email.com	None	2025-09-06 19:22:18.133	1	Dr. Alice	Miller	Cardiology	555-1122	alice
5	5	Daniel	Brown	1990-03-25	M	555-6543	654 Maple St, Springfield, IL	daniel.brown@email.com	Migraine	2025-09-06 19:22:18.133	1	Dr. Alice	Miller	Cardiology	555-1122	alice
6	6	Olivia	Martinez	1980-07-10	F	555-9876	987 Cedar St, Springfield, IL	olivia.martinez@email.com	Arthritis	2025-09-06 19:22:18.133	1	Dr. Alice	Miller	Cardiology	555-1122	alice
7	7	James	Wilson	1965-01-14	M	555-1357	159 Elm St, Springfield, IL	james.wilson@email.com	Heart disease	2025-09-06 19:22:18.133	1	Dr. Alice	Miller	Cardiology	555-1122	alice
8	8	Sophia	Moore	1995-04-05	F	555-2468	753 Oak St, Springfield, IL	sophia.moore@email.com	None	2025-09-06 19:22:18.133	1	Dr. Alice	Miller	Cardiology	555-1122	alice
9	9	David	Taylor	1983-09-20	M	555-3579	369 Pine St, Springfield, IL	david.taylor@email.com	High cholesterol	2025-09-06 19:22:18.133	1	Dr. Alice	Miller	Cardiology	555-1122	alice
10	10	Isabella	Anderson	2005-12-01	F	555-4680	741 Birch St, Springfield, IL	isabella.anderson@email.com	None	2025-09-06 19:22:18.133	1	Dr. Alice	Miller	Cardiology	555-1122	alice
11	1	John	Doe	1985-05-15	M	555-1234	123 Elm St, Springfield, IL	john.doe@email.com	Hypertension	2025-09-06 19:22:18.133	2	Dr. Bob	Williams	Neurology	555-2233	bob
12	2	Jane	Smith	1992-08-22	F	555-5678	456 Oak St, Springfield, IL	jane.smith@email.com	Asthma	2025-09-06 19:22:18.133	2	Dr. Bob	Williams	Neurology	555-2233	bob
13	3	Michael	Johnson	1975-02-10	M	555-8765	789 Pine St, Springfield, IL	michael.johnson@email.com	Diabetes	2025-09-06 19:22:18.133	2	Dr. Bob	Williams	Neurology	555-2233	bob
14	4	Emily	Davis	2000-11-30	F	555-4321	321 Birch St, Springfield, IL	emily.davis@email.com	None	2025-09-06 19:22:18.133	2	Dr. Bob	Williams	Neurology	555-2233	bob
15	5	Daniel	Brown	1990-03-25	M	555-6543	654 Maple St, Springfield, IL	daniel.brown@email.com	Migraine	2025-09-06 19:22:18.133	2	Dr. Bob	Williams	Neurology	555-2233	bob
16	6	Olivia	Martinez	1980-07-10	F	555-9876	987 Cedar St, Springfield, IL	olivia.martinez@email.com	Arthritis	2025-09-06 19:22:18.133	2	Dr. Bob	Williams	Neurology	555-2233	bob
17	7	James	Wilson	1965-01-14	M	555-1357	159 Elm St, Springfield, IL	james.wilson@email.com	Heart disease	2025-09-06 19:22:18.133	2	Dr. Bob	Williams	Neurology	555-2233	bob
18	8	Sophia	Moore	1995-04-05	F	555-2468	753 Oak St, Springfield, IL	sophia.moore@email.com	None	2025-09-06 19:22:18.133	2	Dr. Bob	Williams	Neurology	555-2233	bob
19	9	David	Taylor	1983-09-20	M	555-3579	369 Pine St, Springfield, IL	david.taylor@email.com	High cholesterol	2025-09-06 19:22:18.133	2	Dr. Bob	Williams	Neurology	555-2233	bob
20	10	Isabella	Anderson	2005-12-01	F	555-4680	741 Birch St, Springfield, IL	isabella.anderson@email.com	None	2025-09-06 19:22:18.133	2	Dr. Bob	Williams	Neurology	555-2233	bob
21	1	John	Doe	1985-05-15	M	555-1234	123 Elm St, Springfield, IL	john.doe@email.com	Hypertension	2025-09-06 19:22:18.133	3	Dr. Charlie	Brown	Orthopedics	555-3344	char
22	2	Jane	Smith	1992-08-22	F	555-5678	456 Oak St, Springfield, IL	jane.smith@email.com	Asthma	2025-09-06 19:22:18.133	3	Dr. Charlie	Brown	Orthopedics	555-3344	char
23	3	Michael	Johnson	1975-02-10	M	555-8765	789 Pine St, Springfield, IL	michael.johnson@email.com	Diabetes	2025-09-06 19:22:18.133	3	Dr. Charlie	Brown	Orthopedics	555-3344	char
24	4	Emily	Davis	2000-11-30	F	555-4321	321 Birch St, Springfield, IL	emily.davis@email.com	None	2025-09-06 19:22:18.133	3	Dr. Charlie	Brown	Orthopedics	555-3344	char
25	5	Daniel	Brown	1990-03-25	M	555-6543	654 Maple St, Springfield, IL	daniel.brown@email.com	Migraine	2025-09-06 19:22:18.133	3	Dr. Charlie	Brown	Orthopedics	555-3344	char

Query executed successfully.

UZZAL\SQLSERVER (16.0 RTM) UZZAL\UZZAL2200 (66) HospitalManagementSystem 00:00:00 100 rows

This query retrieves the Cartesian product of the Patients and Doctors tables. It combines every row in the Patients table with every row in the Doctors table, resulting in all possible combinations of patients and doctors. This type of query is typically not used in practice because it creates a large number of results, but it can be useful in certain situations where you want to check all possible combinations.

Nested query

-- Operation: Get all patients who have an appointment on a specific date

```
SELECT * FROM Patients
WHERE patient_id IN (
  SELECT patient_id
  FROM Appointments
  WHERE appointment_date = '2024-11-20'
```

```
);
```

Results Messages										
	patient_id	first_name	last_name	date_of_birth	gender	contact_number	address	email	medical_history	created_at
1	1	John	Doe	1985-05-15	M	555-1234	123 Elm St, Springfield, IL	john.doe@email.com	Hypertension	2025-09-06 19:22:18.133

Create the stored procedure

```
CREATE PROCEDURE GetAllPatients
AS
BEGIN
  SELECT * FROM Patients;
END;
```

EXEC GetAllPatients;

	patient_id	first_name	last_name	date_of_birth	gender	contact_number	address	email	medical_history	created_at
1	1	John	Doe	1985-05-15	M	555-1234	123 Elm St, Springfield, IL	john.doe@email.com	Hypertension	2025-09-06 19:22:18.133
2	2	Jane	Smith	1992-08-22	F	555-5678	456 Oak St, Springfield, IL	jane.smith@email.com	Asthma	2025-09-06 19:22:18.133
3	3	Michael	Johnson	1975-02-10	M	555-8765	789 Pine St, Springfield, IL	michael.johnson@email.com	Diabetes	2025-09-06 19:22:18.133
4	4	Emily	Davis	2000-11-30	F	555-4321	321 Birch St, Springfield, IL	emily.davis@email.com	None	2025-09-06 19:22:18.133
5	5	Daniel	Brown	1990-03-25	M	555-6543	654 Maple St, Springfield, IL	daniel.brown@email.com	Migraine	2025-09-06 19:22:18.133
6	6	Olivia	Martinez	1980-07-10	F	555-9876	987 Cedar St, Springfield, IL	olivia.martinez@email.com	Arthritis	2025-09-06 19:22:18.133
7	7	James	Wilson	1965-01-14	M	555-1357	159 Elm St, Springfield, IL	james.wilson@email.com	Heart disease	2025-09-06 19:22:18.133
8	8	Sophia	Moore	1995-04-05	F	555-2468	753 Oak St, Springfield, IL	sophia.moore@email.com	None	2025-09-06 19:22:18.133
9	9	David	Taylor	1983-09-20	M	555-3579	369 Pine St, Springfield, IL	david.taylor@email.com	High cholesterol	2025-09-06 19:22:18.133
10	10	Isabella	Anderson	2005-12-01	F	555-4680	741 Birch St, Springfield, IL	isabella.anderson@email.com	None	2025-09-06 19:22:18.133

This stored procedure is created to retrieve all records from the Patients table. Stored procedures are used to group multiple SQL statements into a single unit, which can be executed with a single command. This is useful for repetitive tasks.

Execute the Stored Procedure to Get All Patient Records

This command executes the GetAllPatients stored procedure, which retrieves all records from the Patients table. Stored procedures are executed using the EXEC command.

Create a Parameterized Stored Procedure to Fetch Patients by Medical History

```
CREATE PROCEDURE GetPatientsByHistory (@history NVARCHAR(255))
AS
BEGIN
    SELECT * FROM Patients WHERE medical_history = @history;
END;
EXEC GetPatientsByHistory @history = 'Hypertension';
```

This parameterized stored procedure is created to fetch patients based on a specific medical history. The @history parameter allows you to dynamically specify the medical condition, and the procedure retrieves all patients who have that medical history. This command executes the GetPatientsByHistory stored procedure, passing in the value 'Hypertension' as the medical history. It retrieves all patients who have been diagnosed with hypertension

Trigger to Update Billing Status When an Appointment Status is Updated

```
CREATE TRIGGER UpdateBillingStatus
ON Appointments
AFTER UPDATE
AS
BEGIN
    IF UPDATE(status)
    BEGIN
        UPDATE Billing
        SET payment_status = 'Pending'
        WHERE appointment_id IN (SELECT appointment_id FROM inserted);
    END
END;
```

	bill_id	patient_id	appointment_id	total_amount	payment_status	payment_date	insurance_provider	created_at
1	1	1	1	100.00	Pending	NULL	Blue Cross	2025-09-06 19:42:13.560

This trigger automatically updates the billing status to "Pending" whenever the status of an appointment is updated. The AFTER UPDATE clause ensures that the trigger runs after an update operation on the Appointments table. The INSERTED table contains the updated rows, and the trigger checks for changes in the status column.

5. ER Diagram

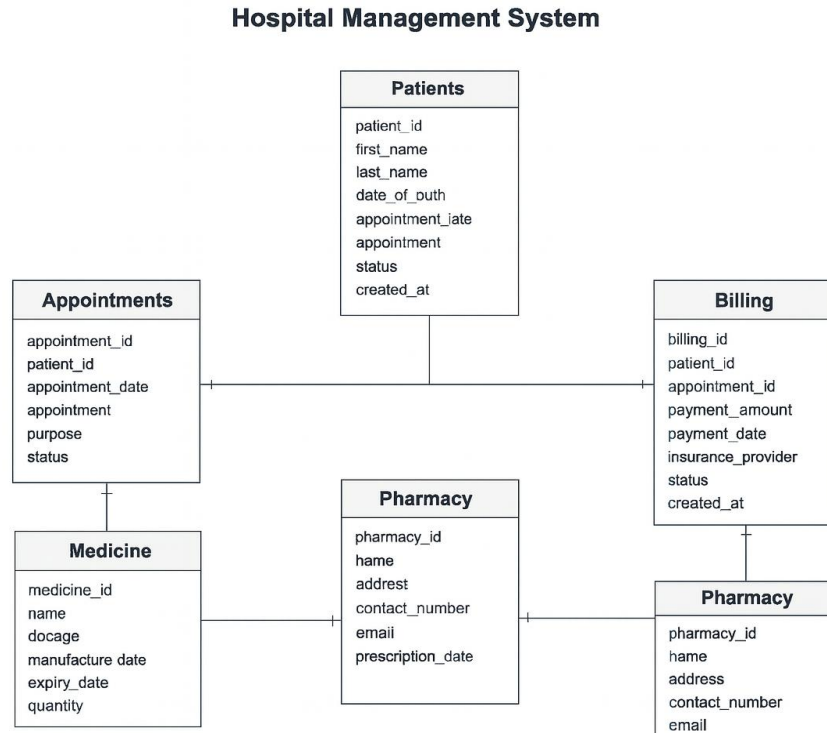


Figure 01: ER diagram of Hospital Management System

6. Results and Discussion

The implemented Hospital Management System allows hospitals to handle patient data, doctor schedules, billing, and medicine inventory more efficiently. Queries can retrieve meaningful insights such as the total number of patients, patients grouped by gender, or those with specific medical histories. Stored procedures simplify operations like checking if a patient exists or fetching all patients with a particular disease. Triggers automate billing updates, ensuring consistency without manual intervention.

7. Conclusion

The Hospital Management System provides a structured and reliable solution to the problems faced by hospitals in managing operations manually. By integrating patient details, appointments, medical records, billing, staff, and pharmacy into one system, it ensures consistency, accuracy, and efficiency. The database supports automation, reporting, and real-time updates, which ultimately improves patient care and hospital performance.

